

Informatics II, Spring 2026, Solution 4

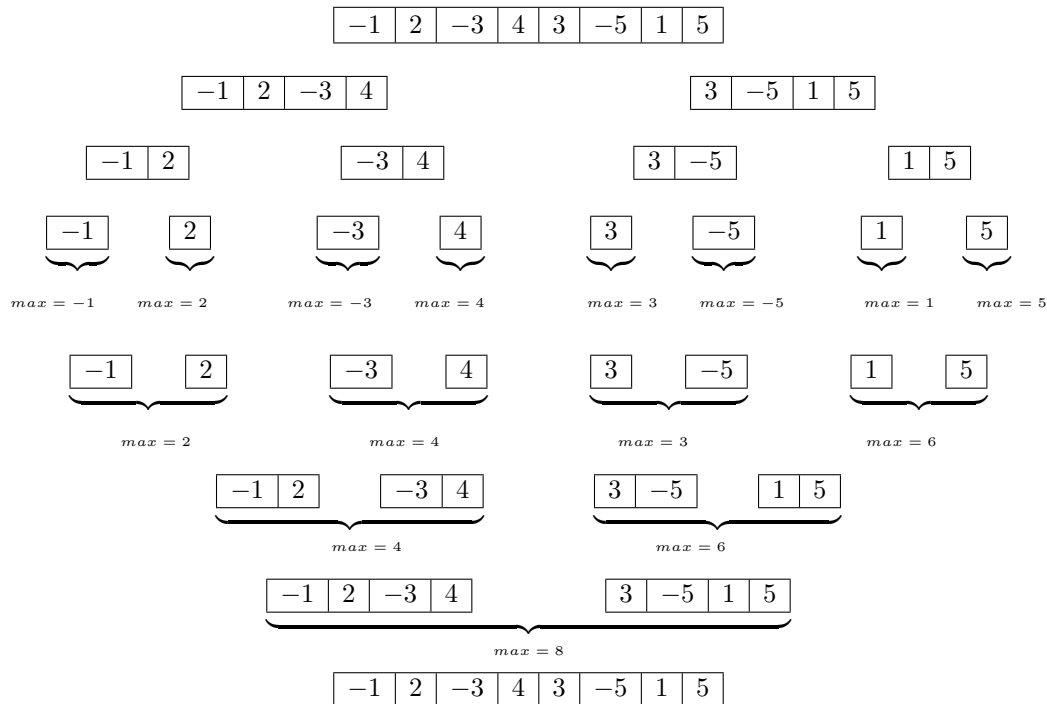
Task 1

Divide and conquer: binary search halves the sorted array each step, choosing the half that may contain the target. code in c:

```
1 #include <stdio.h>
2 int bs(int a[], int n, int x){
3     int l=0, r=n-1, m;
4     while(l<=r){
5         m=(l+r)/2;
6         if(a[m]==x) return m;
7         if(a[m]<x) l=m+1;
8         else r=m-1;
9     }
10    return -1;
11 }
12 int main(){
13     int A[]={1,3,5,7,9,11,13}, n=7, x=7;
14     printf("%d\n", bs(A,n,x));
15 }
```

Task 2

2.1



2.2

```

1  int maxOverlapArraySum(int arr[], int l, int m, int r) {
2      int sum = 0, i=0;
3      int left_sum = -50000;
4      for ( i = m; i >= l; i--) {
5          sum = sum + arr[i];
6          if (sum > left_sum) {
7              left_sum = sum;
8          }
9      }
10     sum = 0;
11     i=0;
12     int right_sum = -50000;
13     for (i = m+1; i <= r; i++) {
14         sum = sum + arr[i];
15         if (sum > right_sum) {
16             right_sum = sum;
17         }
18     }
19     return left_sum + right_sum;
20 }
21
22 int maxSubArraySum(int arr[], int l, int r) {
23     if (l == r)
24         return arr[l];
25     int m = (l + r)/2;
26     int leftArraySum = maxSubArraySum(arr, l, m);
27     int rightArraySum = maxSubArraySum(arr, m+1, r);
28     int overlapArraySum = maxOverlapArraySum(arr, l, m, r);
29     if (leftArraySum > rightArraySum && leftArraySum > overlapArraySum) {
30         return leftArraySum;
31     } else if (rightArraySum > leftArraySum && rightArraySum > overlapArraySum) {

```

```
32     return rightArraySum;
33 }
34 return overlapArraySum;
35 }
36 int max_sum = maxSubArraySum(arr, 0, n-1);
```

2.3

Recurrence: $T(n) = 2T(n/2) + \Theta(n)$

Asymptotic Complexity : $\Theta(n \log n)$

Task 3

3.1 A good guess is $O(2^n)$.

```
1 int T(int n, int c1) {  
2  
3     if (n <= 1) {  
4         return 1;  
5     }  
6     int t = 2*T(n-1, c1) + c1;  
7     printf("%d\n", t);  
8     return t;  
9 }
```

3.2

Guess

$$T_a = O(2^n)$$

We need to show that $T_a(n) \leq k2^n - b$, for some constant k and b .

Base case

$$n = 1$$

$$T_a(1) = 1 \leq k2^1 - b = 2k - b. \text{ This is true for } k \geq (b+1)/2.$$

Inductive step

$$\begin{aligned} T_a(n) &= 2T_a(n-1) + c_1 \\ &\leq 2(k2^{n-1} - b) + c_1 \\ &= k2^n - 2b + c_1 \\ &\leq k2^n - b \end{aligned}$$

This is true for $b \geq c_1$.

We have two constraints that need to be satisfied for our proof to work. The definition of the O notation allows us to choose any constant. Thus, by choosing the constants $b = c_1$ and $k = (b+1)/2$ we satisfy the constraints and show that the property $T_a(n) \in O(2^n)$ is true.

Task 4

Starting with n we compute the recurrence step by step moving backwards.

First, we need to compute $T(n/2) = 2T(\frac{n}{4}) + \frac{n}{2}$.

Next, we back substitute the value of $T(n/2)$ in $T(n)$:

$$\begin{aligned} T(n) &= 2T(n/2) + n \\ &= 2(2T(\frac{n}{4}) + \frac{n}{2}) + n \quad \text{back substitute} \\ &= 2^2T(\frac{n}{2^2}) + 2n \quad \text{simplify expression} \end{aligned} \tag{1}$$

If we continue to proceed with the backward substitution, we will observe a pattern that forms.

$$\begin{aligned}
 T(n) &= 2^2 T\left(\frac{n}{2^2}\right) + 2n \\
 &= 2^3 T\left(\frac{n}{2^3}\right) + 3n \\
 &= 2^4 T\left(\frac{n}{2^4}\right) + 4n \\
 &\dots \\
 &= 2^k T\left(\frac{n}{2^k}\right) + kn
 \end{aligned} \tag{2}$$

Continue until we reach the base case $T\left(\frac{n}{2^k}\right) = T(1) = 1$. Note, we reach the base case after exactly $\log_2(n)$ steps ($2^{\log_2(n)} = n$). Thus, $k = \log_2(n)$

Now, replace k in the equation above.

$$\begin{aligned}
 T(n) &= 2^{\log_2 n} T(1) + n \log_2(n) \\
 &= cn + n \log_2(n) \\
 &= O(n \log_2(n))
 \end{aligned} \tag{3}$$

We used $2^{\log_2(n)} = n$ and replaced $T(1)$ with a constant c . The term $n \log_2(n)$ dominates, thus, we can drop cn in the big O notation.

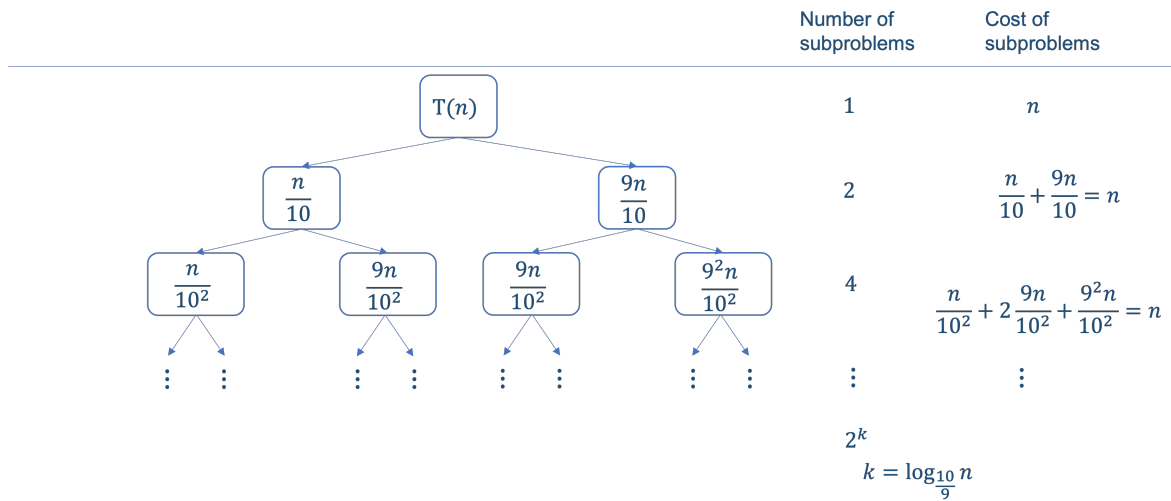
Task 5

5.1

$$T(n) = \begin{cases} 1, & \text{if } n = 1 \\ T\left(\frac{n}{10}\right) + T\left(\frac{9n}{10}\right) + n, & \text{otherwise} \end{cases} \tag{4}$$

At each step, the algorithm performs two recursive calls and does work equal to n . The base case is $n \leq 1$.

5.2



In order to find the tree height, we need to find the longest path from root to a leaf node. At every branch, n is multiplied by either $\frac{9}{10}$ or $\frac{1}{10}$. Thus, the branches in which n is multiplied by $\frac{9}{10}$ will form the longest path.

We observe that the problem size shrinks as follows: $(\frac{9}{10})^0 n, (\frac{9}{10})^1 n, (\frac{9}{10})^2 n, \dots, (\frac{9}{10})^k n$.

The size of the problem at the last level is $(\frac{9}{10})^k n$. Given the base case, we can now solve for k .

$$\begin{aligned} \left(\frac{9}{10}\right)^k n &= 1 \\ \left(\frac{9}{10}\right)^k &= \frac{1}{n} \\ k &= \log_{\frac{10}{9}}(n) \end{aligned} \tag{5}$$

Accounting for the root node, the tree height is $\log_{\frac{10}{9}}(n) + 1$.

Now, we need to calculate the incurred cost.

At each level, we incur costs equal to n (see the tree above).

Thus, the total cost is $n \log_{\frac{10}{9}}(n)$ and we can state that $T(n) \in O(n \log_{\frac{10}{9}}(n))$

Task 6

- 6.1**
- $a = 2, b = 4, f(n) = \sqrt{n} = n^{(\frac{1}{2})}, \frac{1}{2} = \log_4(2)$, thus, case 2 applies.
 - solution: $T(n) \in O(\sqrt{n} \log_4(n))$
- 6.2**
- $a = 12, b = 8, f(n) = n^3$, case 3 applies because the work at the root dominates.
 - solution: $T(n) \in O(n^3)$